

Contents

Recap on Data Types and Structures	6
Code Interpretation	6
Question 1	6
Question 2	6
Code Writing	6
Question 1	6
Question 2	7
Code Fixing	7
Question 1	7
Question 2	7
Multiple Choice Questions	7
Solutions	9
Code Interpretation Solutions	9
Solution 1	9
Solution 2	9
Code Writing Solutions	9
Solution 1	9
Solution 2	9
Code Fixing Solutions	10
Solution 1	10
Solution 2	10
Multiple Choice Answers	10
Recap on Control Structures	11
Code Interpretation	11
Question 1	11
Question 2	11
Code Writing	11
Question 1	11
Question 2	12
Code Fixing	12
Question 1	12

Question 2	12
Multiple Choice Questions	12
Solutions	14
Code Interpretation Solutions	14
Solution 1	14
Solution 2	14
Code Writing Solutions	14
Solution 1	14
Solution 2	15
Code Fixing Solutions	15
Solution 1	15
Solution 2	15
Multiple Choice Answers	15
Recap on Functions	16
Code Interpretation	16
Question 1	16
Question 2	16
Code Writing	16
Question 1	16
Question 2	17
Code Fixing	17
Question 1	17
Question 2	17
Multiple Choice Questions	17
Solutions	19
Code Interpretation Solutions	19
Solution 1	19
Solution 2	19
Code Writing Solutions	19
Solution 1	19
Solution 2	19
Code Fixing Solutions	20
Solution 1	20
Solution 2	20
Multiple Choice Answers	20
Recap Object Oriented Programming	21
Code Interpretation	21
Question 1	21
Question 2	21
Code Writing	22
Question 1	22
Question 2	22

Code Fixing	22
Question 1	22
Question 2	22
Multiple Choice Questions	23
Solutions	24
Code Interpretation Solutions	24
Solution 1	24
Solution 2	24
Code Writing Solutions	24
Solution 1	24
Solution 2	24
Code Fixing Solutions	25
Solution 1	25
Solution 2	25
Multiple Choice Answers	25
Recap on Numerical Computing	26
Code Interpretation	26
Question 1	26
Question 2	26
Code Writing	26
Question 1	26
Question 2	27
Code Fixing	27
Question 1	27
Question 2	27
Multiple Choice Questions	27
Solutions	29
Code Interpretation Solutions	29
Solution 1	29
Solution 2	29
Code Writing Solutions	29
Solution 1	29
Solution 2	30
Code Fixing Solutions	30
Solution 1	30
Solution 2	30
Multiple Choice Answers	30
Recap on Data Manipulation with Pandas	31
1. Code Interpretation Exercises	31
2. Code Writing Exercises	32
3. Code Fixing Exercises	32
4. Multiple Choice Questions	33

Solutions	34
Recap on Input & Output Operations with Pandas	36
Code Interpretation	36
Question 1	36
Question 2	36
Code Writing	36
Question 1	36
Question 2	37
Code Fixing	37
Question 1	37
Question 2	37
Multiple Choice Questions	37
Solutions	39
Code Interpretation Solutions	39
Solution 1	39
Solution 2	39
Code Writing Solutions	39
Solution 1	39
Solution 2	39
Code Fixing Solutions	40
Solution 1	40
Solution 2	40
Multiple Choice Answers	40
Recap on Time Series Analysis	41
Code Interpretation	41
Question 1	41
Question 2	41
Code Writing	42
Question 1	42
Question 2	42
Code Fixing	42
Question 1	42
Question 2	42
Multiple Choice Questions	42
Solutions	44
Code Interpretation Solutions	44
Solution 1	44
Solution 2	44
Code Writing Solutions	44
Solution 1	44
Solution 2	45
Code Fixing Solutions	45

Solution 1	45
Solution 2	45
Multiple Choice Answers	45
Recap on Machine Learning	46
Code Interpretation	46
Question 1	46
Question 2	46
Code Writing	47
Question 1	47
Question 2	47
Code Fixing	47
Question 1	47
Question 2	47
Multiple Choice Questions	48
Solutions	49
Code Interpretation Solutions	49
Solution 1	49
Solution 2	49
Code Writing Solutions	49
Solution 1	49
Solution 2	50
Code Fixing Solutions	50
Solution 1	50
Solution 2	50
Multiple Choice Answers	51

Recap on Data Types and Structures

Code Interpretation

Question 1

Analyze the following code. What does it do, and what is the expected output?

```
transactions = [100, -50, 200, -30, 400]
transactions.append(-100)
print("Updated Transactions:", transactions)
```

Question 2

What will the following code output? Explain why.

```
account_balances = {
    "1234567890": 1000,
    "0987654321": 2500,
    "1122334455": 750
}
print("Balance of account 1234567890:", account_balances["1234567890"])
```

Code Writing

Question 1

Write a function that takes a list of transactions (positive for deposits, negative for withdrawals) and returns the total balance.

Question 2

Write a function that takes a dictionary of account balances and returns a list of account numbers with balances above a specified threshold.

Code Fixing

Question 1

The following code attempts to add a new key-value pair to a nested dictionary, but it has an error. Fix it.

```
d = {"name": "Alice", "details": {"age": 30}}
d["details"].append({"country": "France"})
print(d)
```

Question 2

The following code has a logical error. Fix it to correctly update the dictionary.

```
d = {"name": "Alice", "age": 30}
if "birthday":
    d["age"] += 1
print(d["age"])
```

Multiple Choice Questions

1. What data type is returned by `type(True)`?
 - a) `int`
 - b) `str`
 - c) `bool`
 - d) `float`
2. Which of the following is a mutable data structure?
 - a) `tuple`
 - b) `list`
 - c) `str`
 - d) `int`
3. What does the following code output?
`"Hello World".split()`

- a) ["Hello World"]
 - b) ["Hello", "World"]
 - c) ["H", "e", "l", "l", "o", " ", "W", "o", "r", "l", "d"]
 - d) None of the above
4. How do you access the value associated with the key "name" in a dictionary d?
- a) d.get(name)
 - b) d.name
 - c) d["name"]
 - d) get(d, "name")
5. What is the result of the following code?
- ```
s = set([1, 2, 3, 2, 1])
print(len(s))
```
- a) 3
  - b) 5
  - c) 2
  - d) 1
-



# Solutions

## Code Interpretation Solutions

### Solution 1

The code creates a list of transactions and appends -100 to the list. The final output will be:

Updated Transactions: [100, -50, 200, -30, 400, -100]

### Solution 2

The code defines a dictionary `account_balances` and retrieves the balance of the account with the key "1234567890". The output will be:

Balance of account 1234567890: 1000

---

## Code Writing Solutions

### Solution 1

```
def calculate_balance(transactions):
 return sum(transactions)

transactions = [100, -50, 200, -30, 400]
print("Total Balance:", calculate_balance(transactions))
```

### Solution 2

```
def accounts_above_threshold(account_balances, threshold):
 return [acc for acc, balance in account_balances.items() if balance > threshold]

account_balances = {
 "1234567890": 1000,
```

```
 "0987654321": 2500,
 "1122334455": 750
 }
 print(accounts_above_threshold(account_balances, 1000))
```

---

## Code Fixing Solutions

### Solution 1

```
d = {"name": "Alice", "details": {"age": 30}}
d["details"]["country"] = "France"
print(d)
```

### Solution 2

```
birthday = True
if birthday:
 d["age"] += 1
print(d["age"])
```

---

## Multiple Choice Answers

1. Answer: c
2. Answer: b
3. Answer: b
4. Answer: c
5. Answer: a

# Recap on Control Structures

## Code Interpretation

### Question 1

Analyze the following code. What does it do, and what is the expected output?

```
balance = 1000
monthly_payment = 200

while balance > 0:
 balance -= monthly_payment
 print(f"Remaining balance: ${balance}")
```

---

### Question 2

What will the following code output? Explain why.

```
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
 if fruit == 'banana':
 continue
 print(fruit)
```

---

## Code Writing

### Question 1

Write a `while` loop to calculate the sum of all integers from 1 to 10.

---

## Question 2

Write a `for` loop that prints all the even numbers between 1 and 20.

---

## Code Fixing

### Question 1

The following code aims to calculate the factorial of a number using a `while` loop, but it has an error. Fix it.

```
n = 5
factorial = 1

while n > 0:
 factorial = factorial * n
print(factorial)
```

---

### Question 2

The following code attempts to iterate over a dictionary, but it has an error. Fix it.

```
d = {"name": "Alice", "age": 30}
for key, value in d:
 print(key, value)
```

---

## Multiple Choice Questions

1. Which keyword is used to terminate a loop prematurely?

- a) `continue`
- b) `break`
- c) `exit`
- d) `stop`

2. What will the following code print?

```
for i in range(5):
 if i == 3:
 break
 print(i)
```

- a) 0 1 2 3

- b) 0 1 2
  - c) 0 1 2 4
  - d) 0 1
3. What is the purpose of the `continue` statement in a loop?
- a) To exit the loop prematurely.
  - b) To skip the rest of the current iteration and continue with the next.
  - c) To restart the loop.
  - d) To stop the loop and execute the `else` block.
4. What will the following code output?
- ```
count = 0
while count < 3:
    count += 1
    print(count)
```
- a) 1 2 3
 - b) 0 1 2
 - c) 0 1 2 3
 - d) 1 2 3 4
5. How can you loop through both keys and values in a dictionary?
- a) `for key in d:`
 - b) `for value in d.values():`
 - c) `for key, value in d.items():`
 - d) None of the above
-

Solutions

Code Interpretation Solutions

Solution 1

The code decrements the balance by 200 until it reaches zero or below. The output will be:

```
Remaining balance: $800
Remaining balance: $600
Remaining balance: $400
Remaining balance: $200
Remaining balance: $0
```

Solution 2

The code prints all fruits except “banana” because the `continue` statement skips the iteration when the condition is met. The output will be:

```
apple
cherry
```

Code Writing Solutions

Solution 1

```
sum = 0
n = 1
while n <= 10:
    sum += n
    n += 1
print(sum) # Output: 55
```

Solution 2

```
for num in range(1, 21):  
    if num % 2 == 0:  
        print(num)
```

Code Fixing Solutions

Solution 1

The while loop lacks a decrement for n. Corrected code:

```
n = 5  
factorial = 1  
  
while n > 0:  
    factorial *= n  
    n -= 1  
print(factorial)  # Output: 120
```

Solution 2

The dictionary items() method is needed for unpacking key-value pairs. Corrected code:

```
d = {"name": "Alice", "age": 30}  
for key, value in d.items():  
    print(key, value)
```

Multiple Choice Answers

1. Answer: b
2. Answer: b
3. Answer: b
4. Answer: a
5. Answer: c

Recap on Functions

Code Interpretation

Question 1

Analyze the following code. What does it do, and what will it print for `generate_series(5)`?

```
def generate_series(n):  
    result = []  
    for i in range(1, n + 1):  
        result.append(i ** 2)  
    return result  
print(generate_series(5))
```

Question 2

What does the following code do, and what will it return for `process_data([1, 2, 3, 4, 5])`?

```
def process_data(data):  
    return [x * 2 for x in data if x % 2 == 0]
```

Code Writing

Question 1

Write a function called `square` that takes a number as input and returns its square.

Question 2

Write a function called `find_max` that takes a list of numbers and returns the largest number in the list.

Code Fixing

Question 1

The following code throws an error. Fix it to work as intended.

```
def divide_numbers(a, b):  
    if b = 0:  
        return "Cannot divide by zero"  
    return a / b
```

Question 2

The following function contains a logical error. Fix it.

```
def multiply_list(numbers):  
    product = 0  
    for num in numbers:  
        product *= num  
    return product
```

Multiple Choice Questions

1. What is the correct way to define a function in Python?
 - a) `function my_function():`
 - b) `def my_function():`
 - c) `my_function def():`
 - d) `def: my_function()`
2. What does the `return` statement do in a function?
 - a) Prints a value.
 - b) Ends the function and optionally sends a value back.
 - c) Loops back to the start of the function.
 - d) Declares a function parameter.
3. How do you call a function named `calculate` with arguments 3 and 4?

- a) `calculate(3, 4)`
- b) `calculate 3, 4`
- c) `call calculate(3, 4)`
- d) `calculate{3, 4}`

4. What is the output of the following code?

```
def add(a, b):  
    return a + b  
print(add(2, 3))
```

- a) 2
- b) 5
- c) (2, 3)
- d) None

5. Can a function have no parameters?

- a) Yes.
 - b) No, functions must always take parameters.
 - c) Only if it is a built-in function.
 - d) Only if it uses `*args`.
-

Solutions

Code Interpretation Solutions

Solution 1

The function `generate_series` generates a list of squares of integers from 1 to `n`. It uses a `for` loop to iterate from 1 to `n`, computes the square of each number, and appends it to the result list. For `generate_series(5)`, the output is:

```
[1, 4, 9, 16, 25]
```

Solution 2

The function `process_data` uses list comprehension to create a new list. It doubles each number in the input list `data` if the number is even. For `process_data([1, 2, 3, 4, 5])`, the output is:

```
[4, 8]
```

Code Writing Solutions

Solution 1

```
def square(num):  
    return num ** 2
```

Solution 2

```
def find_max(numbers):  
    return max(numbers)
```

Code Fixing Solutions

Solution 1

```
def divide_numbers(a, b):  
    if b == 0: # Corrected syntax for comparison  
        return "Cannot divide by zero"  
    return a / b
```

Solution 2

```
def multiply_list(numbers):  
    product = 1 # Initialize with 1, not 0  
    for num in numbers:  
        product *= num  
    return product
```

Multiple Choice Answers

1. Answer: b
2. Answer: b
3. Answer: a
4. Answer: b
5. Answer: a

Recap Object Oriented Programming

Code Interpretation

Question 1

Analyze the following class definition. What does the `greet` method do, and what will the output of `p.greet()` be?

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hello, {self.name}!"

p = Person("Alice")
print(p.greet())
```

Question 2

What does the following code do, and what will be the output for `c.display()`?

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2
    def display(self):
        return f"The area of the circle is {self.area()}"

c = Circle(5)
print(c.display())
```

Code Writing

Question 1

Write a class `Rectangle` with:

- A constructor that takes the width and height.
- A method `area` that calculates the rectangle's area.

Question 2

Write a class `BankAccount` with:

- A constructor that initializes the account balance.
- A method `deposit` to add money to the account.
- A method `withdraw` to subtract money, ensuring the balance doesn't go negative.

Code Fixing

Question 1

The following code contains an error. Fix it.

```
class Student:
    def __init__(self, name, age):
        name = name
        age = age

s = Student("John", 20)
print(s.name, s.age)
```

Question 2

The following code has a logical issue. Fix it.

```
class Square:
    def __init__(self, side):
        self.side = side
    def perimeter(self):
        return self.side + self.side

s = Square(4)
print(s.perimeter())
```

Multiple Choice Questions

1. What is the purpose of the `__init__` method in a Python class?

- a) It is the destructor method.
- b) It initializes an instance of the class.
- c) It is called only once in the program.
- d) It is a built-in method for overloading operators.

2. How do you access an instance variable `name` of an object `obj`?

- a) `name.obj`
- b) `obj::name`
- c) `obj.name`
- d) `obj(name)`

3. What will the following code output?

```
class MyClass:
    def __init__(self, x):
        self.x = x
obj = MyClass(10)
print(obj.x)
```

- a) 10
- b) None
- c) Error
- d) `obj.x`

4. What does the `self` keyword represent in a Python class?

- a) A reference to the class itself.
- b) A reference to the current instance of the class.
- c) A placeholder for the superclass.
- d) A special variable for private methods.

5. Which of the following methods is used to represent an object as a string?

- a) `__repr__`
 - b) `__str__`
 - c) `__init__`
 - d) `__format__`
-

Solutions

Code Interpretation Solutions

Solution 1

The `Person` class defines a `greet` method that returns a greeting message with the person's name. For `p.greet()`, the output is:

Hello, Alice!

Solution 2

The `Circle` class calculates the area of a circle and displays it using the `display` method. For `c.display()` with `radius = 5`, the output is:

The area of the circle is 78.5

Code Writing Solutions

Solution 1

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
```

Solution 2

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def deposit(self, amount):
```



```
        self.balance += amount
def withdraw(self, amount):
    if amount <= self.balance:
        self.balance -= amount
    else:
        print("Insufficient funds")
```

Code Fixing Solutions

Solution 1

```
class Student:
    def __init__(self, name, age):
        self.name = name # Fixed by using self
        self.age = age

s = Student("John", 20)
print(s.name, s.age)
```

Solution 2

```
class Square:
    def __init__(self, side):
        self.side = side
    def perimeter(self):
        return 4 * self.side # Fixed logical issue

s = Square(4)
print(s.perimeter()) # Output: 16
```

Multiple Choice Answers

1. Answer: b
2. Answer: c
3. Answer: a
4. Answer: b
5. Answer: b

Recap on Numerical Computing

Code Interpretation

Question 1

Analyze the following code. What does it do, and what will be the output?

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
diag = np.diag(matrix)
print(diag)
```

Question 2

What does the following code do? What is the value of `result`?

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.sum(a[:, None] * b, axis=0)
print(result)
```

Code Writing

Question 1

Write a function using NumPy that generates a 2D array where each element at position `(i, j)` is equal to the product of `i` and `j`.

Question 2

Write a function that normalizes a 1D NumPy array so that the sum of its elements equals 1.

Code Fixing

Question 1

The following code contains an error. Fix it.

```
import numpy as np
matrix = np.zeros[3, 3]  # Error in creating the array
print(matrix)
```

Question 2

The following code intends to calculate the row-wise sum of a 2D NumPy array but has a logical issue. Fix it.

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
row_sums = np.sum(matrix)  # Incorrect axis specification
print(row_sums)
```

Multiple Choice Questions

1. Which of the following functions creates an array of zeros?

- a) `np.zeros()`
- b) `np.zeros_like()`
- c) `np.zeros_all()`
- d) `np.empty()`

2. What will the following code output?

```
import numpy as np
arr = np.array([10, 20, 30, 40])
print(arr[1:3])
```

- a) `[10, 20]`
- b) `[20, 30]`
- c) `[30, 40]`
- d) `[20, 30, 40]`

3. How do you calculate the standard deviation of a NumPy array `arr`?
- a) `np.std(arr)`
 - b) `np.mean(arr)`
 - c) `np.var(arr)`
 - d) `np.median(arr)`
4. What does the `np.linspace()` function do?
- a) Creates an array of equally spaced values within a specified interval.
 - b) Generates an array of random values.
 - c) Creates an array of ones.
 - d) Computes the cumulative sum of an array.
5. Which NumPy function calculates the cumulative sum of an array?
- a) `np.sum()`
 - b) `np.cumsum()`
 - c) `np.add()`
 - d) `np.total()`
-

Solutions

Code Interpretation Solutions

Solution 1

The `np.diag` function extracts the diagonal elements of a square matrix. For the given `matrix`, the output is:

```
[1, 5, 9]
```

Solution 2

The code computes a sum involving element-wise products of arrays `a` and `b`, where `a[:, None]` reshapes `a` for broadcasting. The resulting `result` is:

```
[ 4, 10, 18]
```

Code Writing Solutions

Solution 1

```
import numpy as np
def generate_2d_array(rows, cols):
    result = np.zeros((rows, cols), dtype=int)
    for i in range(rows):
        for j in range(cols):
            result[i, j] = i * j
    return result

print(generate_2d_array(4, 4))
```

Solution 2

```
import numpy as np
def normalize_array(arr):
    return arr / np.sum(arr)

print(normalize_array(np.array([1, 2, 3])))
```

Code Fixing Solutions

Solution 1

The error is due to the incorrect use of square brackets instead of parentheses in `np.zeros`. The fixed code is:

```
import numpy as np
matrix = np.zeros((3, 3)) # Use parentheses
print(matrix)
```

Solution 2

The code does not specify the correct axis for row-wise summation. The fixed code is:

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
row_sums = np.sum(matrix, axis=1) # Specify axis=1 for rows
print(row_sums) # Output: [ 6, 15, 24]
```

Multiple Choice Answers

1. Answer: a
2. Answer: b
3. Answer: a
4. Answer: a
5. Answer: b

Recap on Data Manipulation with Pandas

1. Code Interpretation Exercises

1.1 Code Interpretation Question 1

```
import pandas as pd

data = {
    'Product': ['A', 'B', 'C', 'D'],
    'Price': [100, 200, 300, 400],
    'Stock': [10, 20, 15, 25]
}
df = pd.DataFrame(data)

result = df['Price'] * df['Stock']
print(result)
```

Question: Explain what the `result` variable contains and why.

1.2 Code Interpretation Question 2

```
import pandas as pd

data = {
    'Item': ['X', 'Y', 'Z'],
    'Cost': [50, 75, 120],
    'Discount': [0.1, 0.2, 0.05]
}
df = pd.DataFrame(data)

df['Final Cost'] = df['Cost'] * (1 - df['Discount'])
print(df)
```

Question: What does the `Final Cost` column represent, and how is it calculated?

2. Code Writing Exercises

2.1 Code Writing Question 1

Create a `DataFrame` with the following data:

Employee	Department	Salary
John	HR	50000
Jane	HR	60000
Peter	IT	70000
Lucy	IT	80000

Then, calculate the average salary for each department and display the result.

2.2 Code Writing Question 2

Write a program to create the following `DataFrame`:

Product	Price	Quantity
A	10	2
B	15	5
C	8	10

Add a new column `Total Value` that multiplies `Price` by `Quantity`.

3. Code Fixing Exercises

3.1 Code Fixing Question 1

The following code has a bug. Fix the error to make the code run correctly.

```
import pandas as pd

data = {
    'Product': ['A', 'B', 'C'],
    'Price': [100, 200, 300],
    'Stock': [10, 20, 30]
```



```
}  
  
df = pd.DataFrame(data)  
df['Total'] = df.Price * Stock
```

3.2 Code Fixing Question 2

The following code throws an error. Fix it to make the code work as intended.

```
import pandas as pd  
  
data = {  
    'Item': ['X', 'Y', 'Z'],  
    'Cost': [100, 200, 300],  
    'Discount': [0.1, 0.2, 0.3]  
}  
df = pd.DataFrame(data)  
  
discounted = df.Discount / df.Cost
```

4. Multiple Choice Questions

4.1 Question 1

What does the .iloc method do in a DataFrame? - A) Selects rows based on column labels. - B) Selects rows based on their integer position. - C) Selects rows based on a condition. - D) Selects rows based on their data type.

4.2 Question 2

How do you access the first few rows of a DataFrame in Pandas?

- A) df.head()
 - B) df.start()
 - C) df.first_rows()
 - D) df.rows()
-

4.3 Question 3

Which method is used to check for missing values in a DataFrame?

- A) `df.isnull()`
 - B) `df.isna()`
 - C) `df.check_missing()`
 - D) `df.isnan()`
-

4.4 Question 4

What does the following code return?

```
df['Price'] > 100
```

- A) Rows where `Price` is greater than 100
 - B) A Boolean Series indicating where `Price > 100`
 - C) An error because `Price` is a column
 - D) The count of rows with `Price > 100`
-

4.5 Question 5

Which of the following is the correct way to drop a column in Pandas?

- A) `df.drop(columns=['column_name'])`
 - B) `df.remove_column('column_name')`
 - C) `df.delete_column('column_name')`
 - D) `df.delete(['column_name'])`
-

Solutions

1. Code Interpretation Exercises

1.1: The `result` contains the product of `Price` and `Stock` for each row. It results in a `Series` with values `[1000, 4000, 4500, 10000]`.

1.2: The **Final Cost** column represents the price after applying a discount. It is calculated by multiplying **Cost** by $(1 - \text{Discount})$ for each row.

2. Code Writing Exercises

2.1:

```
data = {
    'Employee': ['John', 'Jane', 'Peter', 'Lucy'],
    'Department': ['HR', 'HR', 'IT', 'IT'],
    'Salary': [50000, 60000, 70000, 80000]
}
df = pd.DataFrame(data)
result = df.groupby('Department')['Salary'].mean()
print(result)
```

2.2:

```
data = {
    'Product': ['A', 'B', 'C'],
    'Price': [10, 15, 8],
    'Quantity': [2, 5, 10]
}
df = pd.DataFrame(data)
df['Total Value'] = df['Price'] * df['Quantity']
print(df)
```

3. Code Fixing Exercises

3.1:

```
df['Total'] = df['Price'] * df['Stock']
```

3.2:

```
discounted = df['Cost'] * df['Discount']
```

4. Multiple Choice Questions

- 4.1: B)
- 4.2: A)
- 4.3: B)
- 4.4: B)
- 4.5: A)

Recap on Input & Output Operations with Pandas

Code Interpretation

Question 1

Analyze the following code. What does it do, and what will be the output?

```
df = pd.read_csv("data.csv", usecols=["Name", "Age"])
df = df[df['Age'] > 30]
df.to_csv("filtered_data.csv", index=False)
```

Question 2

What does the following code do? What is the content of the output file?

```
df = pd.read_excel("data.xlsx", sheet_name="Employee Data", usecols=["ID", "Salary"])
df.to_excel("output.xlsx", sheet_name="Filtered Data", index=False)
```

Code Writing

Question 1

Write a code snippet that: - Reads the first 50 rows of a CSV file named `sales.csv`. - Filters the rows where the `Amount` column is greater than 1000. - Saves the filtered data to a new file called `high_sales.csv`.

Question 2

Write a code snippet to: - Read a specific sheet named `Inventory` from an Excel file `products.xlsx`. - Extract only the columns `Product` and `Quantity`. - Save the resulting DataFrame to a new Excel file named `filtered_inventory.xlsx`.

Code Fixing

Question 1

The following code contains an error. Fix it.

```
df = pd.read("transactions.csv")
filtered_df = df[df['Amount'] > 500]
df.to_csv("filtered_transactions.csv")
```

Question 2

The following code intends to write a DataFrame `df` to an Excel file with the sheet name `Summary`, but it produces an error. Fix the code.

```
df.to_excel("output.xlsx", sheet="Summary")
```

Multiple Choice Questions

1. You want to read only the first 10 rows of a CSV file called `data.csv`. Which of the following commands achieves this?
 - a) `pd.read_csv('data.csv', head=10)`
 - b) `pd.read_csv('data.csv', nrows=10)`
 - c) `pd.read_csv('data.csv', skiprows=10)`
 - d) `pd.read_csv('data.csv', rows=10)`
2. When saving a DataFrame to a CSV file, how can you exclude the index column in the output file?
 - a) Use `index=False` in the `to_csv()` method
 - b) Use `columns=False` in the `to_csv()` method
 - c) Use `save_index=False` in the `to_csv()` method
 - d) Use `drop_index=True` in the `to_csv()` method
3. What does the following command do?

```
df = pd.read_csv("data.csv", usecols=["Name", "Age"])
```

- a) Reads the entire `data.csv` file
 - b) Reads only the `Name` and `Age` columns from `data.csv`
 - c) Renames the columns in `data.csv` to `Name` and `Age`
 - d) Drops all rows in `data.csv` except those containing `Name` and `Age`
4. Which argument in the `read_excel()` method specifies the name of the sheet to read?
- a) `sheet`
 - b) `sheet_index`
 - c) `sheet_name`
 - d) `sheet_select`
5. You need to export a `DataFrame` to an Excel file but want to write it to a specific sheet called `Summary`. Which of the following is correct?
- a) `df.to_excel("output.xlsx", sheet_name="Summary")`
 - b) `df.to_excel("output.xlsx", page="Summary")`
 - c) `df.to_excel("output.xlsx", worksheet="Summary")`
 - d) `df.to_excel("output.xlsx", sheet="Summary")`
-

Solutions

Code Interpretation Solutions

Solution 1

- Reads only the `Name` and `Age` columns from `data.csv`.
- Filters rows where `Age` is greater than 30.
- Saves the filtered DataFrame to a new file named `filtered_data.csv`, excluding the index column.

Solution 2

- Reads the sheet `Employee Data` from `data.xlsx` with only `ID` and `Salary` columns.
 - Writes the filtered data to `output.xlsx` under a sheet named `Filtered Data`, excluding the index column.
-

Code Writing Solutions

Solution 1

```
df = pd.read_csv("sales.csv", nrows=50)
filtered_df = df[df['Amount'] > 1000]
filtered_df.to_csv("high_sales.csv", index=False)
```

Solution 2

```
df = pd.read_excel("products.xlsx", sheet_name="Inventory", usecols=["Product", "Quantity"])
df.to_excel("filtered_inventory.xlsx", index=False)
```

Code Fixing Solutions

Solution 1

Corrected Code:

```
df = pd.read_csv("transactions.csv")
filtered_df = df[df['Amount'] > 500]
filtered_df.to_csv("filtered_transactions.csv", index=False)
```

Solution 2

Corrected Code:

```
df.to_excel("output.xlsx", sheet_name="Summary", index=False)
```

Multiple Choice Answers

1. Answer: b
2. Answer: a
3. Answer: b
4. Answer: c
5. Answer: a

Recap on Time Series Analysis

Code Interpretation

Question 1

Analyze the following code. What does it do, and what will be the output?

```
import pandas as pd

# Creating a time series DataFrame
dates = pd.date_range('2023-01-01', periods=10, freq='D')
values = [10, 12, 15, 13, 17, 19, 23, 22, 24, 20]
df = pd.DataFrame({'Date': dates, 'Value': values}).set_index('Date')

# Resampling the data to weekly frequency
weekly_avg = df['Value'].resample('W').mean()
print(weekly_avg)
```

Question 2

What does the following code do? What is the result of the `rolling_avg` variable?

```
# Calculate the rolling average with a window of 3
rolling_avg = df['Value'].rolling(window=3).mean()
print(rolling_avg)
```

Code Writing

Question 1

Write a code snippet that: - Loads a CSV file containing daily stock prices with columns **Date** and **Close**. - Converts the **Date** column to a **datetime** type and sets it as the index. - Filters the data to only include dates in January 2023.

Question 2

Write a code snippet to: - Generate a DataFrame with daily time series data for the month of January 2023. - Add a new column containing the difference between consecutive values.

Code Fixing

Question 1

The following code is intended to calculate monthly averages for a time series. However, it produces an error. Fix it.

```
# Calculate monthly average
monthly_avg = df.resample('M')['Values'].mean()
print(monthly_avg)
```

Question 2

The following code is supposed to calculate the cumulative sum of time series values but has a bug. Fix it.

```
# Calculate the cumulative sum
df['Cumulative'] = df.cumsum()
```

Multiple Choice Questions

1. Which method is used to convert a column to datetime format in pandas?
 - a) `pd.datetime()`
 - b) `pd.to_datetime()`
 - c) `pd.date_convert()`
 - d) `pd.to_date()`

2. What does the `resample('W')` method do?
 - a) Filters data to include only weekends
 - b) Aggregates data at a weekly frequency
 - c) Converts data to a daily frequency
 - d) Calculates weekly differences
 3. Which of the following is NOT a component of time series data?
 - a) Trend
 - b) Seasonality
 - c) Noise
 - d) Classification
 4. Which method is used to calculate the rolling sum for a time series with a window of 7 days?
 - a) `df.rolling(7).sum()`
 - b) `df.resample('7D').sum()`
 - c) `df.rolling(window=7).mean()`
 - d) `df.sum(rolling=7)`
 5. Which of the following commands sets a `Date` column as the index after converting it to datetime format?
 - a) `df = df.set_index(pd.to_datetime('Date'))`
 - b) `df['Date'] = pd.to_datetime(df['Date']); df.set_index('Date', inplace=True)`
 - c) `df.set_index('Date', datetime=True)`
 - d) `df['Date'] = df.to_date(); df.set_index('Date', inplace=True)`
-

Solutions

Code Interpretation Solutions

Solution 1

- Creates a time series DataFrame with dates as the index and values as the data.
- Resamples the data to weekly frequency and calculates the mean value for each week.

Solution 2

- Calculates a rolling average with a window of 3 for the time series.
-

Code Writing Solutions

Solution 1

```
import pandas as pd

# Load and preprocess the CSV file
df = pd.read_csv("stock_prices.csv")
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')

# Filter for January 2023
january_data = df.loc['2023-01']
print(january_data)
```

Solution 2

```
import pandas as pd

# Generate time series data
dates = pd.date_range('2023-01-01', '2023-01-31', freq='D')
values = range(1, 32)
df = pd.DataFrame({'Date': dates, 'Value': values}).set_index('Date')

# Add difference column
df['Difference'] = df['Value'].diff()
print(df)
```

Code Fixing Solutions

Solution 1

Corrected Code:

```
# Calculate monthly average
monthly_avg = df.resample('M')['Value'].mean()
print(monthly_avg)
```

Solution 2

Corrected Code:

```
# Calculate the cumulative sum
df['Cumulative'] = df['Value'].cumsum()
```

Multiple Choice Answers

1. Answer: b
2. Answer: b
3. Answer: d
4. Answer: a
5. Answer: b

Recap on Machine Learning

Code Interpretation

Question 1

Analyze the following code. What does it do, and what will be the output?

```
from sklearn.tree import DecisionTreeClassifier

# Sample data
X = [[1, 2], [3, 4], [5, 6], [7, 8]]
y = [0, 1, 0, 1]

# Initialize and train the model
clf = DecisionTreeClassifier(max_depth=2, random_state=42)
clf.fit(X, y)

# Predict outcomes
predictions = clf.predict([[2, 3], [6, 7]])
print(predictions)
```

- What is the purpose of `max_depth` in the `DecisionTreeClassifier`?
 - What will the `predictions` output be?
-

Question 2

What does the following code do?

```
from sklearn.cluster import KMeans

# Randomly generated data
X = [[1.0, 2.0], [1.5, 1.8], [5.0, 8.0], [8.0, 8.0], [1.0, 0.6], [9.0, 11.0]]
kmeans = KMeans(n_clusters=2, random_state=42)

# Fit the model
```

```
kmeans.fit(X)
print(kmeans.labels_)
```

- What does `kmeans.labels_` represent?
 - What is the significance of `n_clusters`?
-

Code Writing

Question 1

Write a code snippet that: - Loads a dataset from a CSV file using pandas. - Splits the dataset into training and testing sets using an 80-20 split. - Fits a decision tree classifier to the training data and evaluates its accuracy on the test set.

Question 2

Write a code snippet to: - Generate a random dataset with two features and 100 samples. - Apply K-means clustering with three clusters and print the cluster centroids.

Code Fixing

Question 1

The following code is intended to normalize the features of a dataset using `StandardScaler` but has a bug. Fix it.

```
from sklearn.preprocessing import StandardScaler

# Sample data
X = [[1, 2], [3, 4], [5, 6]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform[X]
```

Question 2

The following code is supposed to evaluate a decision tree classifier but has an error. Fix it.

```
from sklearn.metrics import classification_accuracy

# Evaluate model performance
accuracy = classification_accuracy(y_true, y_pred)
print(f"Accuracy: {accuracy}")
```

Multiple Choice Questions

1. Which method is used to split data into training and testing sets in scikit-learn?
 - a) `train_test_split()`
 - b) `train_test()`
 - c) `test_train_split()`
 - d) `split_train_test()`
 2. What does the `random_state` parameter in scikit-learn models control?
 - a) The number of iterations
 - b) The order of data in the input dataset
 - c) The reproducibility of random results
 - d) The size of the dataset
 3. Which of the following is NOT a type of supervised learning?
 - a) Regression
 - b) Classification
 - c) Clustering
 - d) Decision Trees
 4. What does the `fit()` method do in a scikit-learn model?
 - a) Splits the data into training and testing sets
 - b) Trains the model on the provided dataset
 - c) Makes predictions on new data
 - d) Evaluates the model on a test set
 5. Which scikit-learn class is used for clustering data into groups?
 - a) `LinearRegression`
 - b) `DecisionTreeClassifier`
 - c) `KMeans`
 - d) `StandardScaler`
-

Solutions

Code Interpretation Solutions

Solution 1

- The code trains a decision tree classifier with a maximum depth of 2.
- The predictions output will depend on the structure of the trained decision tree.

Solution 2

- `kmeans.labels_` represents the cluster label assigned to each data point in the input dataset.
 - `n_clusters` specifies the number of clusters the algorithm should create.
-

Code Writing Solutions

Solution 1

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("data.csv")
X = df.drop(columns=["target"])
y = df["target"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train and evaluate model
```

```
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Solution 2

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate data
X, _ = make_blobs(n_samples=100, centers=3, n_features=2, random_state=42)

# Apply K-means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
print("Cluster centroids:", kmeans.cluster_centers_)
```

Code Fixing Solutions

Solution 1

Corrected Code:

```
from sklearn.preprocessing import StandardScaler

# Sample data
X = [[1, 2], [3, 4], [5, 6]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Solution 2

Corrected Code:

```
from sklearn.metrics import accuracy_score

# Evaluate model performance
accuracy = accuracy_score(y_true, y_pred)
print(f"Accuracy: {accuracy}")
```

Multiple Choice Answers

1. Answer: a
2. Answer: c
3. Answer: c
4. Answer: b
5. Answer: c